01_Pipeline

EP

2026-07-19

The Pipeline

This pipeline uses the `targets` R package designed to run computationally demanding analysis. This package allows the necessary computation with implicit parallel computing, allowing for a faster run time and more efficiency by skipping tasks that are already up to date [(1)].

The aim of this pipeline is to produce reproducible code that generates a uniform grid across various UK cities. The grid has a number of potential biodiversity and flooding indicators, as well as species occurrence data associated to each grid square. This is then used in baseline modelling, Random Forest, and XGBoost. All results are stored within the pipeline and can be accessed using `tar_load()`.

Dependencies

To run this pipeline, you need to have downloaded and opened `targets` and `tarchetypes`. Other dependencies within the pipeline are listed in the _targets.R file within the `tar_options_set()` line.

This pipeline uses large volumes of data and can be slow to run. This is largely due to the LIDAR and Imperviousness calculations, these can be annotated out for a shorter proof of concept run. To do so, you will just need to remove the 'dtm', 'grid_elev', 'imperv', and 'grid_imperv' from `_targets.R`. These have been highlighted in the script.

Pipeline Structure

This pipeline contains 4 files:

`_targets.R`

This is the overall targets pipeline file which builds an environmental grid by city and models species diversity. Structure:

1. `shared_targets` - This loads raw data files common to all cities.
2. `per_city_targets` - This uses `tar_map()` to iterate over the cities table to clip the raw data to the study boundary, build elevation and imperviousness data from zipped files, and construct a 1km^2 grid. This grid is built up using various predictive variables, producing a `grid_final` for each city. Species data for each grid square is added, and the city grids combined to create `grid_model_all`.
3. Modelling and Reporting - This fits richness, Simpson, Random Forest, and XGBoost models on the combined dataset, then renders `pipeline_report.Rmd` as the final summary report.

`Grid.R`

This file contains the data-processing functions for grid preparation and construction.

Loading and clipping

- `load_vector(path, crs)` is the most simple of these functions, it reads a vector file (the boundary file, in this project) and reprojects it to my project CRS (227700, British National Grid).
- `load_within_boundary(path, crs, boundary)` This reads, reprojects, and filters a vector layer to only features within my study boundary.
- `load_intersecting(path, crs, boundary)` This is similar to the function above but does not rely on the assumption of a single-polygon boundary and uses less memory for large inputs.

Raster Preparation These steps unzip LiDAR and impervious surface tiles, converts them to a VRT, crops to the city boundary, and writes a per city DTM.

Grid Construction This step builds a uniform square 1km grid over the boundary, keeping only cells intersecting with it, and assigning each a `cell_id`.

Grid Feature Engineering

- `add_elevaion` Aggregates the DTM and extracts a mean elevation per cell.
- `add_dominant_risk` Finds the dominant risk band per grid cell by determining the largest overlap. If equally split between risk bands, the highest risk band is taken.
- `add_flood_risk` This calls the above function to run for both ROFSW and ROFRS.
- `add_forest` Joins the average hectares of forest per grid square and concatenated woodland type.
- `add_weighted_climate` Generates an area-weighted mean of the climate variables, precipitation and temperature, per grid cell, intersecting grid cells with climate polygons.
- `add_imperviousness` Aggregates and reprojects the imperviousness raster and extracts a mean value per grid cell.
- `add_river_distance` This calculates the distance from the grid cell centroid to the nearest river.
- `add_greenspace` Determines the percentage of each cell's area covered by greenspaces, as well as a concatenated list of greenspace types.

Output The grid, an sf object, is saved as a GeoPackage, overwriting any existing file on the path. This allows for the most up to date version to always be available.

`Species_Diversity.R`

This file handles the functions related to loading and cleaning species occurrence records, joining them to the grid, and deriving diversity and sampling effort metrics.

Loading and Data Preparation `load_species` reads the species CSV and converts it to points using WGS84 longitude/latitude columns, then converts to the project CRS. Only records strictly within the project boundary are kept (`st_within`). Records are joined to the grid spatially.

Species Incidence and Richness `build_incidence` drops incomplete records and builds a presence/absence matrix (one row per cell_id, one column per species, each with a 1 or 0 value). Rare species (species recorded in less than 3 grid cells) are filtered out at this point to reduce noise in the data. Richness is calculated by summing the presence across species columns per cell.

Species Abundance, Shannon, and Simpson Abundance is similar to incidence but keeps actual counts rather than the presence or absence of a species. `Individual.Count` is summed per species per cell, where missing counts are treated as 1 (an NA in this column doesn't mean there were no species. Existence of a column means the species was spotted but that the total number wasn't recorded.) This calculation is also

restricted to species present in 3 or more cells. Shannon and Simpson diversity are calculated, however only Simpson (adjusted) is used. This is due to a high correlation between the two indices, negating the need for both.

Sampling Effort This records the total number of records (`n_records`), distinct survey days (`n_days`), and distinct number of data providers (`n_providers`). These are used as a proxy for sampling effort and diversity calculations are sensitive to how much the cell was surveyed.

`Baseline_Model.R`

This file contains three functions to generate a null vs full model pair for richness, Shannon, and Simpson diversity. All models use the same explanatory variables: `n_days`, `dtm_elev`, `avg_imperv`, `rofsw_risk`, `rofrs_risk`, `avg_area_ha`, `treeht`, `weighted_precip`, `weighted_temp`, `dist_to_river`, and `pct_greenspace`. The generation of a list with two fitted model objects allows model comparison, e.g.

```
{anova(model$null, model$full)}
```

`fit_richness` uses a negative binomial regression (`glm.nb` from `MASS`), appropriate for this as it uses overdispersed count data.

`fit_shannon` uses a gamma GLM with log link as Shannon index values are continuous, strictly positive, and skewed right.

`fit_simpson` uses a beta regression (`betareg`) which is appropriate since Simpson's index is bounded between 0 and 1. However, the data requires transformation first which has been done using the standard Smithson & Verkuilen (2006) transformation [(2)].

Useful code

Function	Use
`tar_make()`	This runs the entire pipeline. You can specify a run of a specific target within this function.
`tar_manifest()`	This produces a data frame of information about the targets
`tar_visnetwork()`	This displays the dependency graph of the pipeline.
`tar_load()`	This loads objects within the pipeline, e.g. the grid.
`tar_read()`	This reads a targets value from storage.

References

1. William, L. R Documentation (23 September 2020) Targets. Available from: https://www.rdocumentation.org/packages/targets/versions/0.0.0.9000 [Accessed 28 July 2026]
2. Smithson, M. and Verkuilen, J. (2006) A Better Lemon Squeezer? Maximum-Likelihood Regression with Beta-Distributed Dependent Variables. Psychological Methods. 11(54-71) [Accessed 15/07/2026]